OMNIA: Closing the Loop by Leveraging LLMs for Knowledge Graph Completion

Frédéric Ieng*, Soror Sahri*, Mourad Ouzzani†, Massinissa Hammaz*
Salima Benbernou*, Hanieh Khorashadizadeh‡, Sven Groppe‡§, Farah Benamara¶
*LIPADE, Université Paris Cité, Paris, France
†Qatar Computing Research Institute, HBKU, Doha, Qatar
‡Universität zu Lübeck, Germany
§TU Bergakademie Freiberg, Germany
¶Université de Toulouse, IRIT, France
and IPAL-CNRS, Singapore
Emails: {frederic.ieng, soror.sahri, salima.benbernou}@u-paris.fr, massinissa.hammaz@etu.u-paris.fr, mouzzani@hbku.edu.qa,
{hanieh.khorashadizadeh, sven.groppe}@uni-luebeck.de, farah.benamara@irit.fr

***Abstract*—Knowledge Graphs (KGs) are widely used to represent structured knowledge, yet their automatic construction, especially with Large Language Models (LLMs), often results in incomplete or noisy outputs. Knowledge Graph Completion (KGC) aims to infer and add missing triples, but most existing methods either rely on structural embeddings that overlook semantics or language models that ignore the graph's structure and depend on external sources. In this work, we present OMNIA, a two-stage approach that bridges structural and semantic reasoning for KGC. It first generates candidate triples by clustering semantically related entities and relations within the KG, then validates them through lightweight embedding filtering followed by LLM-based semantic validation. OMNIA performs on the internal KG, without external sources, and specifically targets implicit semantics that are most frequent in LLM-generated graphs. Extensive experiments on multiple datasets demonstrate that OMNIA significantly improves F1-score compared to traditional embedding-based models. These results highlight OMNIA's effectiveness and efficiency, as its clustering and filtering stages reduce both search space and validation cost while maintaining high-quality completion.**

I. Introduction

Background. KGs are structured representations of human knowledge, linking entities—real-world objects or abstract concepts—through relations. Their versatility makes them essential in applications such as search, question answering, and intelligent assistants. By structuring knowledge explicitly, KGs enable machines to reason, infer and retrieve information more effectively. However, as they expand in size and complexity, maintaining high quality becomes crucial to ensure reliable performance.

To assess the quality of a KG, various quality dimensions have been proposed [49, 38], with particular emphasis on (i) accuracy–correctness of facts and relationships, as inaccuracies can undermine both trust and utility; (ii) completeness–the extent to which all relevant information is captured, since missing data may lead to overlooked insights; and (iii) consistency–logical coherence by avoiding contradictory statements or violations of predefined rules. In this paper, we focus on inferring triples that are missing from a given KG, a task that pertains to the completeness dimension.

Several methods have been proposed to assess or improve KG's completeness using external text corpora [41], rule mining techniques [20], or embedding-based predictions [4, 34, 31, 44, 11]. These approaches are generally based on fixed ontologies with predefined entities and relations, which limits their ability to handle triples expressed in natural language and implicit semantics, and often ignore the structural relationships encoded within the KG itself.

Large Language Models (LLMs) have recently been applied to KG construction [52, 53, 33, 48] and completion [17, 43, 29, 41, 12], offering new opportunities to address the limitations of traditional KG completion approaches. However, LLMs still struggle to extract or infer triples that capture implicit semantics and often lack sufficient structural context, which reduces the effectiveness of conventional completion methods on LLM-generated KGs.

Motivation. We consider a real-world use case from a collaborative project aiming to analyze the global response to the COVID-19 pandemic using enriched KGs [23]. The following example highlights completion challenges that underscore the need for a more effective, semantic-based approach for KGs. Beyond these challenges, our approach specifically targets semantic issues that are prevalent in LLM-generated KGs, such as handling implicit semantics.

Running example. We illustrate this with a COVID-19 KG [10], generated through an LLM-based extraction process applied to COVID-Fact, a dataset of manually verified facts about the pandemic [24]. The resulting domain-specific KG captures key information covering regulations, policies, statistics and drug-related effects. From the following factual statements:

- f_1: "Remdesivir and chloroquine effectively inhibit the recently emerged 2019-ncov in vitro"
- f_2: "FDA approves the emergency use of chloroquine phosphate and hydroxychloroquine sulfate for treatment

of COVID-19"

- f_3: "A chicago hospital treating severe covid-19 patients with gilead sciences ' antiviral medication remdesivir is seeing recoveries in patients ' symptoms , stat news reports"

using OpenAI GPT4 [10], the following triples were extracted: t_1: (remdesivir, treats, sars-cov-2), t_2: (remdesivir, inhibits, 2019-ncov), and t_3: (chloroquine, inhibits, 2019-ncov).

Following an analysis of the text in COVID-Fact and its factual statements, we noticed that not all factual statements have been extracted and transformed into triples in the KG using the above process. One such example is the triple t_4: *(chloroquine, treats, sars-cov-2)*, which we denote as a missing triple, i.e., a fact entailed by the source data but not extracted by the KG construction method. In our running example, t_4 can be derived from f_2 and is present in the COVID-Fact dataset but was not extracted and is therefore considered a missing triple.

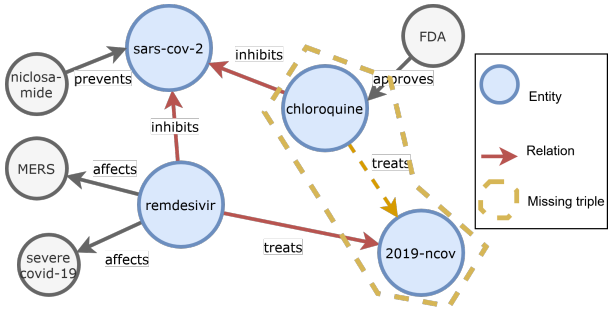


Fig. 1: Example of a missing triple in a LLM-generated knowledge graph.

Figure 1 shows the extracted triples and the missing one. We observe that t_2 and t_3 share the structure *(subject, inhibits, 2019-nCoV)*, suggesting that *remdesivir* and *chloroquine* are semantically related, as they link to the same object through the same relation. A naïve solution to recover missing triples like t_4 would be to generate all possible entity–relation combinations, but this yields numerous non-valid or meaningless candidates. Such brute-force search is inefficient and impractical, highlighting the need for a targeted and efficient strategy to generate valid triple candidates while minimizing the search space. While this example illustrates a missing triple caused by incomplete extraction from source data, in practice, the exact causes of missing triples are often unknown, especially in real-world and LLM-generated KGs. Accordingly, evaluating methods designed to recover such triples requires approximations that allow for controlled and reproducible experimentation, as adopted in our experiments.

Challenges. The motivating example illustrates three key challenges. First, many missing triples are implicitly present in the source text but not extracted, requiring candidate generation methods that can recover such facts efficiently. Second, most existing approaches rely on external knowledge, which is often unavailable or unreliable. In contrast, our work leverages only the internal structure of the KG, ensuring a controlled and schema-consistent completion process. Third, given the large number of candidate triples, an initial filtering step is needed to discard non-valid ones and focus evaluation on the most plausible candidates.

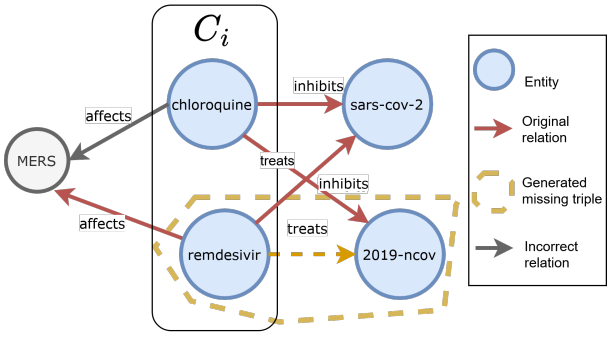


Fig. 2: Example of the Clustering-based triple candidate generation.

Our solution: To address these challenges, we propose OMNIA[1], a two-stage approach for KG completion, as illustrated in Figure 3:

Clustering-based triple candidate generation. To address the challenge of missing triples that are implicit in the text but not explicitly extracted during KG construction, OMNIA groups tail entities and relations into clusters according to shared relational patterns. Within each cluster, new triples are inferred by propagating known (relation, tail) pairs to compatible head entities (cf. Figure 2). For instance, *chloroquine* and *remdesivir* are both connected to *SARS-CoV-2* through the relation *inhibits*, thus belong to the same cluster. Knowing that *chloroquine* treats *2019-nCoV* and *remdesivir* affects *MERS*, OMNIA infers candidate triples: *(chloroquine, affects, MERS)* and *(remdesivir,treats, 2019-ncov)*. However, not all generated triples are valid: the second triple is valid, whereas the first one is not valid. This clustering captures valid yet overlooked triples while reducing exhaustive search, improving both coverage and scalability. It leverages the KG's structural patterns and explicit relation semantics, avoiding dependence on external sources that may be noisy or unavailable, consistent with findings in [47].

Validation of missing candidates. Given the potentially large number of candidate triples, we introduce a two-stage filtering process. A lightweight KG embedding model is first used to eliminate clearly non-valid triples. Then, the most promising candidates are passed to an LLM using different prompting strategies or through a retrieval augmented generation (RAG) system for final validation. This ensures that LLMs are used efficiently while maintaining high precision in the final set of completed triples.

[1]OMNIA means "everything" in Latin, denotes our aim to close the loop by using LLMs not only to construct but also to complete KGs, while remaining applicable to any KG.

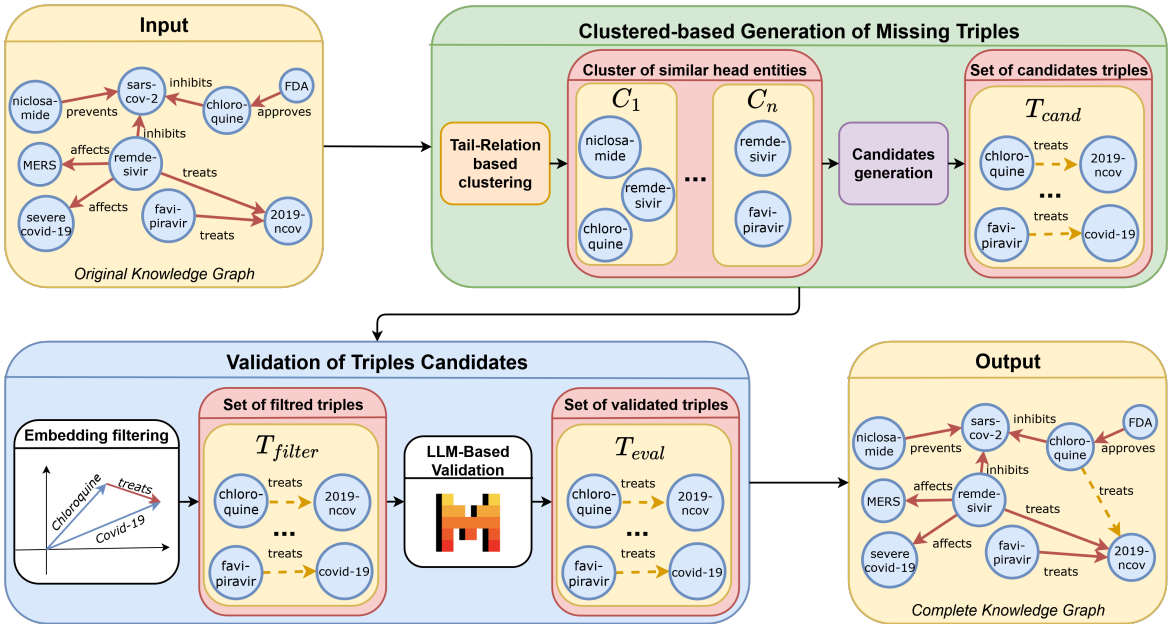


Fig. 3: Overview of OMNIA with its two main steps

Contributions. We summarize our contributions as follows.

- We introduce OMNIA, a novel approach that generates missing triples with a targeted focus on implicit semantics, often overlooked in LLM-generated KGs. While motivated by these cases, OMNIA remains general and effective across diverse KGs. It leverages only the internal graph structure, using structural clustering of entities and relations with similar patterns to efficiently infer relevant missing facts.
- We propose a two-step validation process that combines structural and textual information from the KG without using external sources. The first stage filters non-valid triples using an embedding model, while the second validates the remaining ones in both structured and natural language scenarios using zero-shot, in-context, and retrieval-augmented prompting to assess semantic validity.
- We conduct an extensive experimental evaluation on multiple KGs, including those generated by LLMs. Our proposed approach outperforms strong baselines on triple classification, with F1-score improvements reaching up to 23 percentage points.

The rest of this paper is organized as follows. Section II reviews related work. Section III formalizes the KGC problem. Section IV presents OMNIA with its two main steps. Section V presents the experimental setup and results. Finally, Section VI concludes with future directions.

II. Related Work

Improving the quality of KGs is crucial for maximizing their usability across various applications [13]. To assess their quality, several dimensions are used, including accuracy, completeness, consistency, conciseness, and relevancy [49, 38]. In this paper, we focus on the completeness of KGs.

A. Knowledge Graph Quality Assessment

Several studies have proposed frameworks for assessing and refining KGs. Paulheim [22] classified refinement methods by goal (completion vs. error correction) and data source (internal vs. external). Huaman [13] introduced a curation framework addressing correctness and completeness through 20 quality dimensions, including duplicate resolution and conflict detection. Wang et al. [39] proposed a universal framework that evaluates accuracy, consistency, and trustworthiness through the selection and extraction phases.

B. Knowledge Graph Completion

As part of KG refinement, KGC tasks include three main tasks [3, 27]: *link prediction*, which predicts a missing entity or relation within an incomplete triple; *triple classification* which validate the accuracy of a given triple; and *relation prediction* which identifies the appropriate relation between a pair of entities. In some cases, entity classification is also considered, especially when assigning types to entities supports KG enrichment [21]. To address these tasks:

Traditional embedding based approaches represent entities and relations in vector spaces to capture graph structure [5, 19, 35], but often overlook semantic information.

Language model-based approaches address this limitation by using pretrained models [6]. For example, KG-BERT [46] reformulates triples as textual sequences but struggles with relational disambiguation. StAR [36] and SimKGC [37] improve contextual representations through multitask and contrastive learning, while GPHT [50] introduces Triple Set Prediction (TSP) to generate new triples using subgraph embeddings without relying on external text.

Sequence-to-sequence and training-free approaches redefine KGC as a generation task. KG-S2S [8] and GenKGC [42] improve representation learning with relation-guided demonstrations and entity-aware decoding, while KICGPT [40] enhances efficiency through in-context learning without fine-tuning. KGT5 [26] uses an encoder–decoder transformer with auto-regressive decoding for scalable KG link prediction.

Large language model approaches further integrate semantic reasoning with structural knowledge. KG-LLM [29] transforms triples into natural language to improve reasoning, and MuKDC [18] generates potential missing data filtered by embeddings. Hybrid methods such as KoPA [51] and MPIKGC [43] combine structural embeddings or LLM-generated descriptions to enhance inference.

C. OMNIA's take on the Problem

Unlike traditional embedding-based methods that rely on structural patterns [5, 44, 35], OMNIA combines both the graph's structure and the semantic capabilities of LLMs to capture missing triples, particularly in cases where triples are implicitly entailed. Compared to language model-based methods [46, 36, 37], OMNIA does not rely on external textual corpora. Instead, like GPHT [50], it generates candidates directly from the KG. However, OMNIA specifically targets challenges in LLM-generated KGs, in particular incomplete extraction, which GPHT does not consider. Finally, unlike approaches that use LLMs for end-to-end prediction, OMNIA applies LLMs in a targeted validation step, after filtering structurally unlikely triples, which makes it both efficient and adaptable to real-world KGs with varying characteristics.

Overall, OMNIA adopts a system-level design that integrates structural and semantic reasoning for closed-world KG completion, relying solely on the input KG. In contrast to approaches that integrate semantic reasoning directly into learned representations or hybrid embeddings, OMNIA treats semantic validation as a separate and explicit step, enabling efficient and controlled use of LLMs in closed-world settings.

III. Problem Definition

Let us consider a *Knowledge Graph*, $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, where $\mathcal{E}$ is a set of entities, $\mathcal{R}$ is a set of relations, and $\mathcal{T}$ is a set of triples. Each triple $(h, r, t) \in \mathcal{T}$ consists of a *head entity* $h \in \mathcal{E}$, a relation $r \in \mathcal{R}$, and a *tail entity* $t \in \mathcal{E}$. An example of a triple is *(Probiotics, Prevents, Covid-19 infection)*.

Knowledge Graph Completion (KGC) is the task of recovering triples that are missing from a KG. It may either leverage external sources, such as text corpora, structured databases, or rely solely on the internal structure and content of the graph. This involves two common assumptions [28]: the open-world assumption where new entities or relations not present in the original graph may be introduced, and the closed-world assumption, which limits the completion to combinations of existing entities and relations in the graph.

In this work, we adopt a closed-world setting and focus on KGC operating solely on the KG itself. Unlike link prediction or relation prediction, which assume partially observed triples, our objective is to generate entirely new triples that are *entailed* by the facts but not explicitly present in the graph. In addition, while triple classification typically involves binary plausibility prediction over predefined candidates, our KGC problem extends this by including a dedicated LLM-based validation step to evaluate the generated triples in context, as we present in Section IV-B2.

Missing triples are triples that are not present in the KG, yet are logically entailed by the source text used to construct it. Importantly, the entities and relations involved in such triples are already included in the KG, only the complete statement connecting them is missing. Formally, we define a missing triple in our setting as a triple $(h, r, t) \notin \mathcal{T}$ such that $h, t \in \mathcal{E}$, $r \in \mathcal{R}$, and (h, r, t) is entailed by the source corpus used to generate $\mathcal{G}$. While such missing triples are particularly common in KGs constructed from text using LLMs, our formulation and proposed approach apply to any KG where similar types of incompleteness may arise.

IV. Overview of OMNIA

We introduce OMNIA, a two-stage approach designed to enhance the completeness of KGs, particularly those generated by LLMs. As depicted in Figure 3, the first stage generates missing triple candidates through a cluster-based generation process and the second validates these generated triples through a two-step process consisting of an embedding-based filtering followed by LLM-based validation. By combining these two stages, OMNIA bridges the gap between structural and semantic reasoning in KG completion. Its originality lies in relying on the internal structure and semantics of the KG rather than external textual sources, while still integrating LLMs for selective semantic validation.

OMNIA is designed as a system-level approach, rather than as a new embedding or learning architecture. This design enforces a clear separation between structural inference and LLM-based semantic validation, allowing each stage to focus on a well-defined role and enabling efficient and controlled use of LLMs in closed-world KG completion settings.

To illustrate how our proposed two-stage design operates in practice, we consider the case of the missing triple t_4, form the running example (cf. Section I). In the first stage, entities sharing similar relational contexts (e.g., *chloroquine* and *remdesivir*) are grouped into the same cluster. The underlying intuition is that entities connected through similar relations tend to exhibit a similar behavior, enabling OMNIA to uncover implicit triples like t_4. In the second stage, the inferred triples are filtered and validated, retaining only semantically valid triples, such as t_4. In the rest of the section, we detail each stage of OMNIA and then analyse its complexity.

A. Clustered-based Generation of Missing Triples

This stage aims to infer missing triples by leveraging the internal structural patterns of the KG. OMNIA groups entities and relations into clusters based on their relational similarity, computed from the graph's connectivity patterns

rather than external textual features. This clustering step allows the efficient generation of a high-coverage set of candidate triples by exploiting recurring structural patterns in the graph, while deferring semantic reasoning to the next stage of the approach.

Although this is motivated by patterns frequently observed in LLM-generated KGs, as introduced in the motivating example, the same mechanism generalizes to any KG and can be adapted to different assumptions of missing knowledge beyond implicit semantics, as demonstrated in our experiments. This stage consists of two main steps presented below.

Algorithm 1 Clustering of head entities

Require: A knowledge graph $\mathcal{G} = \{(h_i, r_i, t_i)\}_{i=1}^{n}$
Ensure: A set of clusters $\mathcal{C} = \{\mathcal{C}_1, \mathcal{C}_2, \dots\}$
1: Initialize an empty dictionary: $\mathcal{C} \leftarrow \{\}$
2: **for all** $(h_i, r_i, t_i) \in \mathcal{G}$ **do**
3: $k \leftarrow (r_i, t_i)$ ▷ Tail-relation pair as cluster key
4: **if** $k \in \mathcal{C}$ **then**
5: $\mathcal{C}[k] \leftarrow \mathcal{C}[k] \cup \{h_i\}$
6: **else**
7: $\mathcal{C}[k] \leftarrow \{h_i\}$
8: **end if**
9: **end for**
10: **return** $\mathcal{C}$

1) Entity Clustering: Let us consider $\mathcal{G}$, a KG such as:

$$\mathcal{G} = \{(h_1, r_1, t_1), (h_2, r_2, t_2), \dots, (h_n, r_n, t_n)\}$$

where (h_i, r_i, t_i) represents a triple with h_i representing the head entity, r_i the relation and t_i the tail entity.

The clustering concept in OMNIA is grounded in the intuition that entities appearing in similar structural roles within a KG, namely entities occurring in comparable relational contexts, tend to encode related semantic properties.

To this end, OMNIA groups structurally similar head entities into clusters to generate candidate missing triples.

Algorithm 1 outlines this process. First, for each triple in $\mathcal{G}$, we extract the relation-tail pair (r_i, t_i) (Line 3 in Algorithm 1). If a cluster corresponding to this (r_i, t_i) pair already exists, the head entity h_i is added to that cluster (Lines 4-5). Otherwise, a new cluster is created for the pair (Lines 6-7). The algorithm returns the set of clusters, denoted by C, each corresponding to a distinct relation–tail pair observed in $\mathcal{G}$.

For instance, by applying our clustering to the following triples:
{ *(sars-cov-2, causes, pneumonia)*, *(covid-19, causes, pneumonia)*, *(delta-variant, causes, pneumonia)*, *(remdesivir, treats, covid-19)*, *(remdesivir, inhibit, sars-cov-2)*, *(favipiravir, treats, covid-19)*, *(favipiravir, treats, delta-variant)*}
We obtain the following clusters:
$\mathcal{C}_1$ = {*sars-cov-2, covid-19, delta-variant*}, where all entities share the relation-tail pair *(causes, pneumonia)*;
and $\mathcal{C}_2$ = {*remdesivir, favipiravir*}, where both entities share the relation-tail pair *(treats, covid-19)*. Figure 4 illustrates this example. We note that the same head entity may belong to multiple clusters if it appears in several triples sharing the same relation–tail pair.

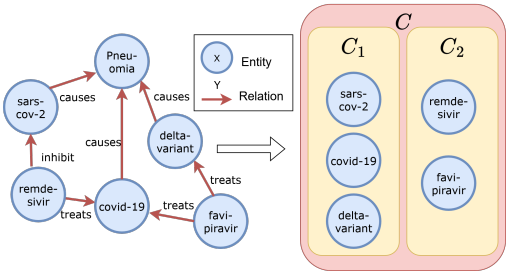


Fig. 4: Example of Entity Clustering

2) Candidate Triples Generation: Let us consider C such as: $C = \{C_1, C_2, \dots, C_K\}$, the set of clusters obtained from the previous clustering step. These clusters are leveraged to systematically generate candidate missing triples, as depicted in Algorithm 2. For each cluster $C_i \in C$, we identify all head entities $h_i \in C_i$ (Line 3). For each of these entities, we collect the relation–tail pairs (r_i, t_i) with which they are associated (Line 4). Candidate triples are generated by combining each head entity within C_i with every relation–tail pair extracted from the same cluster (Line 7). This process results in new triples of the form (h_j, r_i, t_i), which are treated as candidates for missing triples in the KG.

These candidates are subsequently validated in the next stage of OMNIA, as described below. We note that our generation process may lead to fewer candidates in sparse KGs, as it relies on structural patterns present in the KG. We discuss this behavior in our experimental analysis (Section V-D1) and consider extensions to handle sparsity as part of future work.

Algorithm 2 Candidates generation

Require: A set of clusters $\mathcal{C} = \{C_1, C_2, \dots, C_K\}$ ▷ Each C_k contains triples (h_i, r_i, t_i)
Ensure: A set of candidate triples $\mathcal{T}_{\text{cand}}$
1: $\mathcal{T}_{\text{cand}} \leftarrow \emptyset$
2: **for all** $C_k \in \mathcal{C}$ **do**
3: $H_k \leftarrow \{h_i \mid (h_i, r_i, t_i) \in C_k\}$
4: $P_k \leftarrow \{(r_i, t_i) \mid (h_i, r_i, t_i) \in C_k\}$
5: **for all** $h_i \in H_k$ **do**
6: **for all** $(r_i, t_i) \in P_k$ **do**
7: Add (h_i, r_i, t_i) to $\mathcal{T}_{\text{cand}}$
8: **end for**
9: **end for**
10: **end for**
11: **return** $\mathcal{T}_{\text{cand}}$

B. Validation of Candidate Triples

Not all generated triples are necessarily semantically valid. To address this issue, this stage first removes structurally unlikely triples using KG embeddings, and then relies on LLMs to assess the semantic validity of the remaining ones.

1) Embedding-Based Filtering of Candidate Triples: To filter the generated candidates, OMNIA employs TransE [5], a model trained on the original KG to capture latent representations of entities and relations. For each candidate triple (h, r, t), a score is computed in the embedding space, where lower values indicate a higher likelihood that the triple is valid.

For example, given a valid triple $\mathcal{T}_1 = (h_1, r_1, t_1)$ and a non-valid triple $\mathcal{T}_2 = (h_1, r_1, t_2)$, the sum of the head entity embedding h_1 and the relation embedding r_1 is expected to be equal or close to the embedding of the true tail entity t_1, and far from that of the non-valid tail entity t_2. Candidate triples whose distance score falls below a threshold τ are retained, while the others are discarded. This step acts as an efficient pruning mechanism, substantially reducing the candidate space before the subsequent LLM-based validation stage.

We emphasize that TransE serves as a structural validity filter to prune unlikely candidates triples. While its embeddings reflect the observed KG, the model does not explicitly encode assumptions about the distribution of missing triples across relations or entities, relying only on geometric consistency in the embedding space. In other words, we frame TransE as a tool, not a probabilistic model.

2) LLM-Based Validation of Candidate Triples: To check the semantic validity of candidate triples, we employ an LLM as a semantic judge (Algorithm 3). Based on our problem formulation, which focuses on identifying and validating plausible missing triples, this validation step assesses the semantic validity of generated candidate triples given the relational structure of the KG. To address this validation objective, the first design decision concerns the format in which candidate triples are presented to the LLM. Specifically, we consider: (i) triple-based validation, where candidates are evaluated in their original structured form, preserving the relational semantics of the KG and facilitating reasoning over entities and relations; and (ii) sentence-based validation, where each triple is transformed into a natural language statement using LLM, allowing the model to leverage its language understanding capabilities to capture implicit semantic information that may not be explicitly encoded in the graph.

Algorithm 3 LLM evaluation

Require: A set of triple candidate $\mathcal{T}_{cand}$
Ensure: A set of evaluated triples candidate $\mathcal{T}_{eval}$
1: **for** each triple $t \in \mathcal{T}_{cand}$ **do**
2: Ask the LLM to evaluate the candidate
3: **if** t is considered true **then**
4: $\mathcal{T}_{eval} \leftarrow t$
5: **end if**
6: **end for**
7: Return $\mathcal{T}_{eval}$

The second design decision concerns the prompting strategy used during validation, which determines what additional contextual information needs to be provided to the LLM when assessing a candidate triple. In particular, we consider:

- Zero-shot prompting, the LLM is prompted solely with the candidate triple or sentence. This method does not use any form of context nor example and evaluate the triple or sentence as-is. This would show the performance of the plain LLM without additional information.
- Few-shot prompting, the LLM is provided with a small set of related triples that share a common head, relation, or tail with the candidate triple. These triples serve as contextual examples for the evaluation. This context set includes at least one triple that shares each component (head, relation, or tail) with the evaluated triple. By exposing the LLM to related triples, it gains additional context about the candidate through its "relatives", which helps it provide a more accurate validation.
- Retrieval-Augmented Generation (RAG), the LLM receives a top-k set of semantically similar triples retrieved from the embedding space to inform its decision. RAG is a widely-used framework in generative AI that combines the strengths of information retrieval systems with the capabilities of LLMs. It constrains the LLM to respond based on retrieved evidence.By adding semantically similar triples to the a prompt asking about the validity of the candidate triple, the LLM can make a more informed decision using a more relevant context than few-shot prompting.

We also tested several prompt templates and iteratively refined them. Figure 5 depicts each case for the triple-based scenario. For the sentence-based scenario, we use an LLM to transform a triple and the corresponding context into a sentence. This requires explicit prompt constraints, as the LLM may otherwise respond by judging the validity of the triple instead of performing the transformation. Therefore, for evaluation, we explicitly instruct the LLM to only transform the triple into a sentence and avoid any negative or evaluative formulation. This is why for the evaluation, we need to explicitly indicate to the LLM to only transform the triple into a sentence and not use any negative format.

C. Complexity analysis

The clustering step starts by creating an empty dictionary in $O(1)$ time and iterates once over all triples in the KG G. For each triple, it builds a key from the relation–tail pair and performs constant-time dictionary operations to update clusters. Since each triple is processed exactly once, the total time complexity of this step is $O(n)$, where n is the number of triples in the KG. In the candidate generation step, each cluster C_k produces $O(m_k \times p_k)$ combinations, where m_k and p_k denote the number of head entities and relation–tail pairs within the cluster, respectively. The overall cost is therefore $O(\sum_k m_k \times p_k)$. In the worst case, a single cluster contains all n triples, yielding at most n distinct head entities and n distinct relation–tail pairs. Consequently, the candidate generation step admits a worst-case upper bound of $O(n^2)$. The embedding-based filtering is linear, with complexity $O(n + n')$, where n' is the number of generated candidates. Finally, the LLM-based validation evaluates each remaining candidate once,

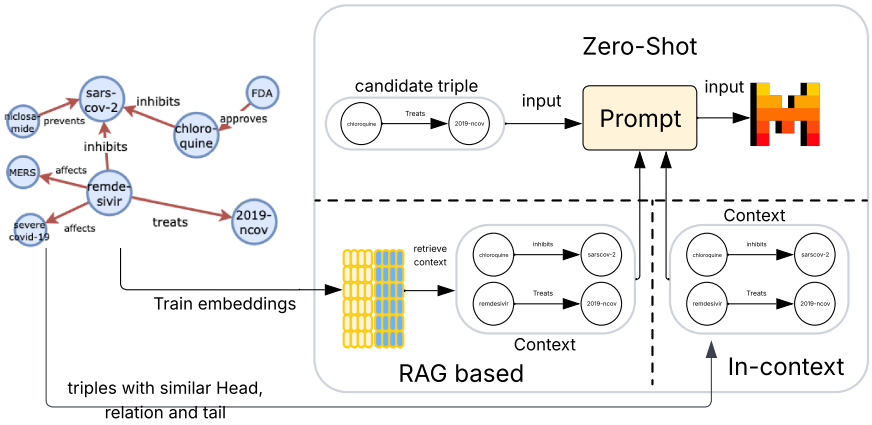


Fig. 5: Triple-based scenario with its three prompting cases for LLM-based validation

resulting in $O(n')$. Overall, the total complexity of OMNIA is $O(n+\sum_k m_k \times p_k + n')$, which is dominated by the candidate generation step.

V. Experimental Evaluation

We conducted comprehensive experiments on OMNIA and addressed the following key questions:

- (Q1) How efficient and effective is OMNIA's candidate generation in identifying relevant missing triples compared to an exhaustive strategy?
- (Q2) How effective is our evaluation model compared to the SOTA baselines from three different classes of methods?
- (Q3) What is the effect of using sentence-based versus triple-based prompting in the LLM-based evaluation?
- (Q4) What are the key parameters that impact the performance of OMNIA's prompting strategies, particularly in RAG?

A. Datasets and Evaluation Metrics

We conducted our experiments on the commonly used datasets for KG completion tasks and KG embeddings evaluation [29, 47, 40] in addition to a KG generated using an LLM as described in our previous work [16] to assess performance in LLM-generated KGs.

- FB15K-237 [32] is a refined version of the FB15K [5] dataset which is originally a subset of Freebase. This version is specially designed to avoid the data leakage problem discovered with the FB15K. The original FB15k contained many inverse relation pairs, which led to inflated performance metrics in KG completion tasks. FB15k-237 resolves this issue by removing such inverse relation pairs. The dataset includes common sense knowledge as well as complex relational facts across a variety of topics.
- CoDEx-M [25] is the medium-scale subset of the CoDEx (Contextualized Decoding for KG Completion) benchmark. It is derived from Wikidata [16] and includes natural names for entities and relations, making it well-suited for language-grounded KG tasks. The dataset is designed to better capture semantic diversity and support contextual reasoning.
- WN18RR[9] is an improved version of WN18 [5], which is a subset of the well-known WordNet dataset. The original WN18 dataset was derived from WordNet but was later found to include a large number of test triples that were simple inverses of triples present in its training split (i.e., the training portion of the WN18 benchmark dataset).
- Socio-Economic is a KG generated using LLMs [16]. It encodes semantic relationships between concepts extracted from economic, geopolitical, and policy documents. This KG is intentionally sparse, offering a clearer view of performance differences among baseline methods. Since this dataset is private and was not part of the training data for the LLMs used in its construction, we consider it private, ensuring that the models had no prior exposure to its content.

Since LLM-based processing is time-consuming, we evaluate the LLM validation stage of our method on a sample of 500 generated candidates for each of the previously mentioned datasets. Table I reports the statistics of these datasets. The table presents statistics for the Covid-Fact dataset [10] used in our motivating example. Due to its limited size, this dataset was not considered in the main experimental evaluation.

To assess the performance of OMNIA, we adopt standard metrics commonly used in binary classification tasks, as our formulation of triple classification follows this setting. Specifically, we report the following metrics: accuracy, precision, recall, and F1-score. We define the standard terms as follows:

Dataset	Relations	Entities	Triples
Socio-economic	17,175	33,563	64,417
CoDEx-M	49	16,759	60,000
FB15K-237	29	12,993	59,270
WN18RR	11	40,943	93,003
Covid-Fact	28	1416	908

TABLE I: Statistics of the datasets used in our experiments

(i) TP (True Positives) refer to correctly identified missing triples, (ii) TN (True Negatives) to correctly identified non-missing triples, (iii) FP (False Positives) to non-missing triples incorrectly classified as missing, and (vi) FN (False Negatives) to missing triples incorrectly classified as non missing.

Ground Truth. To compute the evaluation metrics, we simulate missing triples by removing uniformly at random 20% of the triples from each dataset, without conditioning on entities or relations. The remaining 80% form the observed graph used for candidate generation, while the removed triples are treated as valid but unobserved facts to be recovered. This protocol serves as a controlled proxy for real-world incompleteness, where valid triples may be absent due to extraction failures or limited observations. To ensure robustness, we repeat the random removal five times and report averaged results. Although random removal does not explicitly model all forms of implicit semantics found in LLM-generated KGs, it captures the key challenge of inferring true missing triples from existing structure and context.

B. Baselines

We compare OMNIA against three main classes of baseline methods:

- **KGE-based:** We consider four representative KG embedding models: TransE [4], DistMult [44], ComplEx [34], and RotatE [31]. For each model, we automatically select a classification threshold that maximizes the F1 score on a validation set.
- **PLM-based:** We include KG-BERT [45], a widely used model for triple classification tasks that leverages pre-trained language models. KG-BERT reformulates KG triples as natural language sequences and uses BERT to model their plausibility, enabling it to capture rich contextual and semantic information beyond traditional embedding-based methods.
- **LLM-based:** For LLM based-method, we compare our method OMNIA to MuKDC [18] by evaluating MuKDC using the same evaluation protocol as OMNIA. We implemented MuKDC with a local Llava model using the Ollama library.

C. Implementation and Detail Settings

We use the Mistral-7B model [14] in all experiments. In the RAG setting, embeddings are computed with MPNet [30], and FAISS [15] is used to retrieve the most similar reference triples.

The experimental evaluation was carried out on the Grid'5000 platform [2], on the Graffiti node of the Nancy cluster. This node is equipped with an Intel Xeon Silver 4110 CPU, a GeForce RTX 2080 Ti GPU, and 128 GB of RAM. An exception was made for the exhaustive search of candidate triples, which required more memory and was therefore executed on the Grosminet node within the same cluster. Grosminet features an Intel Xeon Gold 6240L CPU, 6144 GB of RAM, and no GPU.

In the validation step of OMNIA, we first filter the generated candidate triples before LLM-based evaluation. For this, we used PyKEE[1] to implement TransE [5]. To select the optimal filtering threshold, we evaluated four possible values on a validation set: median+0.5 , mean +0.5, median - 0.5 and mean -0.5. The threshold yielding the highest F1-score was retained for use during evaluation. In the RAG case of the LLM-based validation, we used the LangChain[7] framework to manage both retrieval and language generation. To ensure semantically meaningful chunks, we employed the SemanticChunker alongside HuggingFaceEmbeddings[30]. The resulting embeddings were indexed using FAISS[15], enabling efficient similarity-based retrieval during the generation process.

OMNIA uses Mistral-7B by default alongside LangChain, for all LLM-based prompting cases. This model offers a strong balance between performance, efficiency, and scalability, making it well-suited for our validation scenarios.

For the KG embedding (KGE) baselines, we rely on PyKEEN to implement TransE, DistMult, ComplEx, and RotatE. For KG-BERT, we used the official implementation available on GitHub[2]. All baselines were evaluated on the same test set, which includes 500 triples per dataset, evenly distributed between true and false missing triples. In addition, the KGE and PLM-based methods were provided with a validation set consisting of 250 triples, evenly split between positive and negative examples, for selecting the optimal classification threshold. All models were trained on the same set of candidate triples generated by OMNIA, ensuring a consistent and fair comparison across methods.

We now present the results of our experiments in the next subsections. All experiments are repeated five times using different random splits, and the reported results are averaged across runs.

D. Results

1) OMNIA's Triple Generation: In this section, we answer (Q1) related to the efficiency and effectiveness of OMNIA's candidate generation to identify relevant missing triples. We evaluate our proposed approach prior to any downstream filtering or validation. To provide a meaningful comparison, we consider an exhaustive triple generation strategy. Although we did not execute the exhaustive approach in full for all the datasets due to computational constraints, we estimate, on a per-dataset basis, the number of missing triples that would not be generated as a result of discarding certain entities and relations during the experiment setup.

[2]https://github.com/yao8839836/kg-bert

dataset	UE_O	UE_G	UR_O	UR_G	nb_candidates	nb_missings	TP	nb_miss_brut	nb_gen_brut
CoDEx-M	16759	16431	49	48	9047869	11996	70.65%	95.98%	12958932528
Socio-economic	33563	29860	17175	14710	6494589	12347	9.6%	55.76%	13115724316000
FB15K237	12993	12610	29	29	4885007	11854	49.08%	96.12%	4611350900

TABLE II: Comparison of dataset statistics and generation performance. U: unique , E: entities , R: relations , O: original dataset size, G: Left for generation , TP_OMNIA: % of missing triples successfully generated by OMNIA, TP_brut:% of missing triples successfully generated by an exhaustive search, gen_brut: Total number of candidate triples generated by exhaustive search.

The comparative results are presented in Table II. Although our approach generates fewer candidate triples than an exhaustive generation strategy, it offers a favorable trade-off between scalability and coverage. OMNIA prioritizes computational efficiency and scalability, focusing on high-quality candidates rather than brute-force enumeration. To underscore the impracticality of exhaustive generation, we executed it on a sample of the CoDEx-M dataset with the workload distributed across 10 processes. This run required nearly two hours and consumed approximately 2.6 terabytes of RAM. In contrast, OMNIA completes the same task in just 10 minutes on a single process, well within the limits of 8 gigabytes of RAM.

For instance and as presented in Table II, in the CodEx-M sample, the number of unique entities decreased from 16,759 (UE_O) to 16,431 (UE_G), and relations from 49 (UR_O) to 48 (UR_G) after dropping a set of triples used as missing triples for evaluation. As a result, an exhaustive search would generate $16431 \times 16431 \times 48 = 12958932528$ triples, among which 95.98% of the missing triples are generated (TP_brut). In comparison, OMNIA generates 9,047,869 candidate triples; approximately 1,400 times fewer than the exhaustive search, while still generating 70.65% of the missing triples (TP_OMNIA). This experiment demonstrates OMNIA's ability to drastically reduce the candidate space while still successfully generating a substantial portion of the known missing triples.

To better understand the generation performance on sparse versus dense knowledge graphs, we extended the experiment on the socio-economic dataset. Specifically, we created a denser subset, referred to as *Socio2*, containing 12,000 triples in which each entity and relation appears at least three times. OMNIA's generation performance improved from 9.6% on the original sparse dataset to 30% on Socio2. We further evaluated OMNIA on the full CodEx-M and CodEx-S datasets, where it successfully generated over 97% and 98.3% of the missing triples, respectively. These results confirm that OMNIA is particularly effective in dense KGs, where entities and relations occur with greater frequency.

The Socio-economics dataset is a sparse KG where structural patterns are less frequent. Thus, the clustering-based candidate generation produces fewer valid candidates, which may result in lower recall before the validation stage. This behavior mainly concerns the candidate generation step, while the LLM-based validation remains effective once candidates are available. Potential mitigation strategies for sparse graphs are discussed as future work in Sectio VI.

2) OMNIA's Candidate Filtering: As presented previously, filtering plays a crucial role by reducing the number of candidate triples presented to the LLM-based evaluation. It aims to retain only the most promising triples from the large set generated during the first stage, thereby improving computational efficiency without sacrificing too much recall. Table III shows the performance of this filtering step across different datasets. In the table, we report the reduction ratio, the total number of missing triples, the number of true candidates before filtering (TC), and the number of true candidates after filtering (TCF). For the CodEx-M sample, OMNIA initially generates 9 Million candidates that contain 8,476 triples that are actual missing triples (TC). After filtering, the candidates are reduced by 71.08% while introducing a loss in TC also reducing it to 5,024 triples (TCF). The total number of missing triples in this case is 11,997. On the FB15K237 dataset, we observe a 41.76% reduction in candidates, also introducing a loss, from 5,818 (TC) to 3,836 (TCF), in a total of 11,854 missing triples. The same trend is seen on the sparse Socio-economic dataset, where the number of candidates was reduced by 70% for a 34% loss of TC. While the filtering step does result in the loss of some valid triples, the overall computational gain is substantial. Despite a filtering accuracy of around 60%, this step remains beneficial, as it significantly reduces the load for LLM-based evaluation. Further improvements, such as better embedding techniques or refined training, are left as future work.

Dataset	Red. Ratio	Missing	TC	TCF
CoDEx-M	71.08%	11997	8476	5024
Socio-economic	70%	12347	1186	607
FB15K237	41.76%	11854	5818	3836

TABLE III: Filtering performance over different datasets

3) Effectiveness of OMNIA compared to baselines: In this section, we answer question Q2 related to the effectiveness of OMNIA's evaluation model compared to existing baseline methods. We evaluate the performance of OMNIA's LLM-based evaluation on the task of triple classification using standard metrics: accuracy, precision, recall and F1-score. We present the F1-score in Table IV, as it provides a comprehensive summary of the model's performance across all metrics. As discussed earlier, we evaluate OMNIA against representative baselines, including knowledge graph embedding (KGE) models (TransE, DistMult, ComplEx, RotatE), the PLM-based KG-BERT, and the recent LLM-based method MuKDC. OMNIA's evaluation component is assessed under

both sentence-based and triple-based prompting strategies, using three prompting cases: zero-shot, in-context and retrieval-augmented generation (RAG).

Overall, OMNIA demonstrates strong performance across all datasets. In particular, the RAG-based variant of OMNIA consistently achieves the highest F1-scores on FB15k-237 (0.86), CoDEx-M (0.91), and WN18RR (0.87), outperforming all baseline models. These gains highlight the benefit of retrieving relevant context to support triple validation by the LLM. For example, OMNIA outperforms TransE on FB15k-237 (F1: 0.86 vs. 0.74) and CoDEx-M (F1: 0.91 vs. 0.68), and outperforms DistMult on WN18RR (F1: 0.87 vs. 0.77). On the Socio-Economic dataset, OMNIA performs at a comparable level to the baselines, with an F1-score of 0.68, while the best KGE baseline, DistMult, achieves 0.74 and KG-Bert achieves 0.73.

The LLM-based baseline MuKDC has low overall F1-scores across all datasets, with high recall and low precision. This behavior can be attributed to its aggressive candidate generation strategy, which prioritizes coverage over accuracy, leading to a very high recall but poor precision. In contrast, OMNIA maintains a better balance between recall and precision, resulting in substantially higher overall performance.

These results highlight the strength of our evaluation strategy and confirm the added value of using LLMs with context retrieval for the triple classification task. Since the removal is uniformly random, the removed triples include both cases that can be recovered from structural cues and cases that cannot. Using standard benchmarks also shows that our method generalizes beyond LLM-generated KGs. Moreover, OMNIA outperforms state-of-the-art structural methods such as TransE and DistMult, which rely on graph structure but not semantics. This shows that our approach is better at capturing semantic signals beyond pure structural patterns.

E. Effect of Sentence-based vs. Triple-based Prompting

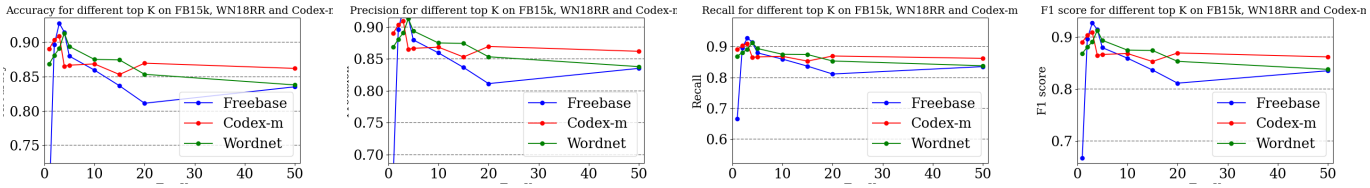
In this section, we answer question Q3 that investigates how the the representation of the candidate impact its evaluation by the LLM. Specifically, we compare the two scenarios in OMNIA's LLM-based evaluation, (i) candidate triples expressed in natural language sentences (OMNIA Sentences), and (ii) structured triples (OMNIA Triples). LLMs are primarily trained to process natural language sentences rather than structured triples. Consequently, representing a candidate triple as a sentence is expected to result in better performance.

The results, presented in Table IV, show that OMNIA sentences variant outperforms OMNIA triples variant across CoDEx-M and Socio-Economic, regardless of the prompting strategy (zero-shot, in-context, or RAG), confirming the benefit of sentence-based validation in these settings. For example, on CodEx-M, both accuracy and F1-score improved by over 5% using a sentence representation. However, on WN18RR, OMNIA triples yield better results. This can be attributed to the dataset's unique entity and relation representations (e.g., "X.nn.02"), which hinder the LLM's ability to correctly interpret triples transformed into sentences. For instance, in the WN18RR dataset, the incorrect candidate triple (*theophrastaceae.n.01 , _member_meronym, pistacia.n.01*) was classified as False by the LLM (Mistral_7B [14]) when given the triple. However, when it was reformulated into the natural language sentence "*A member of the Theophrastaceae is a pistachio tree*", the model revised its judgment and classified it as True. This underscores the importance of supporting both representations to enhance the generalizability of the solution across datasets. However, on WN18RR, OMNIA triples yield better results. This can be attributed to the dataset's unique entity and relation representations (e.g., "X.nn.02"), which hinder the LLM's ability to correctly interpret triples transformed into sentences. For instance, in the WN18RR dataset, the non-valid candidate triple (*theophrastaceae.n.01 , _member_meronym, pistacia.n.01*) was classified as False by the LLM (Mistral_7B [14]) when given the triple. However, when it was reformulated into the natural language sentence "*A member of the Theophrastaceae is a pistachio tree*", the model revised its judgment and classified it as True. This underscores the importance of supporting both representations to enhance the solution's generalizability across datasets.

Category	Model	FB15k-237	CoDEx-M	WN18RR	Socio-Economic
KGE	TransE	0.74	0.682	0.68	0.72
	DistMult	0.6709	0.674	0.77	**0.737**
	ComplEx	0.6661	0.665	0.67	0.66
	RotatE	0.7285	0.662	0.75	0.717
PLM-based	KG-Bert	0.57	0.542	0.62	<u>0.727</u>
LLM-based	MuKDC	0.192	0.273	0.229	0.224
OMNIA Sentences	Zero-shot	0.68	0.68	0.63	0.59
	In-context	0.65	0.69	0.66	0.66
	RAG	<u>0.85</u>	**0.91**	0.72	0.678
OMNIA Triples	Zero-shot	0.75	0.62	0.74	0.579
	In-context	0.61	0.65	<u>0.83</u>	0.649
	RAG	**0.86**	<u>0.84</u>	**0.87**	0.584

TABLE IV: Performance (F1-score) comparison of OMNIA against baselines across four datasets. **Bold** indicates the best per dataset; <u>underlined</u> indicates the second-best. OMNIA (with its different variants) performs the best across all datasets with a big margin (12 percentage points for FB15k-237, 23.6 for CoDEx-M, and 10 for WN18RR) except "Socio-Economic", where it ranks third.

Impact of Prompt Templates. We also experimented with several prompt templates to transform triples into sentences. We found that LLMs are prone to rewriting semantically non valid triples into seemingly valid sentences if not explicitly instructed otherwise. This behavior is problematic, as the evaluation relies on preserving the intended meaning of the original triples. To illustrate this issue, we compare a simple prompt that lacks detailed instructions with an explicit prompt that clearly guides the model to preserve the original semantics of the triple. The simple prompt used is:



(a) Accuracy (b) Precision (c) Recall (d) F1 score

Fig. 6: Scores for different top-K on FB15K-237, WN18RR, CoDEx-M and Socio-economic

Simple prompt:
Transform the following triple into a sentence: {head} {relation} {tail}
Answer: Give the sentence.

This prompt provides no constraint on whether the model should maintain the validity of the triple, as explained above. To address this issue, we design the following explicit prompt, which gives clear, step-by-step instructions to avoid unintended rewriting.

Explicit prompt:
Your task is to only transform the triple into a sentence, no matter if it is valid or not.
A triple has a head, a relation, and a tail.
Transform the following triple: `{head}` `{relation}` `{tail}` as is.
Do not rephrase non-valid facts or use negative constructions.
Answer: Give the sentence.

We evaluate these two prompts on a sample of 500 candidate triples generated from the CoDEx-M dataset. The results in Table V show that without explicit instructions, the LLM incorrectly rewrites 8% of the triples, altering their original meaning. With an explicit prompt, it consistently preserves semantics, achieving 100% correct transformations in this dataset.

Method	Valid trans.	Valid trans. ratio
Simple prompt	462	0.92
Explicit prompt	500	1

TABLE V: Results of simple and explicit prompts for sentence transformation on 500 candidates triples from CoDEx-M dataset

F. Hyperparameter Analysis

In this section, we answer question Q4, by analyzing the effect of the hyperparameter top-k in OMNIA's prompting strategies, particularly in the RAG setting. In a RAG framework, the top-k determines the number of similar triples retrieved from the KG that will be then added as a context for the prompt presented to the LLM. We evaluate how varying this parameter affects performance on the triple-based scenario.

We experimented with top-k values in $\{1, 2, 3, 4, 5, 10, 15, 20, 50\}$ and computed our metrics, namely accuracy, precision, recall, and F1-score for datasets WN18RR, FB15k-237 and CoDEx-M. For example, on CoDEx-M, we observed that setting top-k = 50 introduces many irrelevant triples, making it harder for the model to select the correct answer. In contrast, top-k = 2 yields mostly relevant triples, improving accuracy. As shown in Figure 6, the F1-score peaks at top-k = 3 across all datasets. It drops significantly for top-k = 1 compared to top-k = 2 and top-k = 3. Moreover, for values above 5, F1-scores dropped substantially due to the inclusion of less relevant context. We conclude from this experiment that the optimal range for top-k lies between 2 and 4. Based on these results, we selected top-k = 2 as the optimal setting for our experiments.

G. Generalizability

We evaluate OMNIA's performance across diverse Knowledge Graphs and types of missing triples to assess its generalizability beyond implicit semantic incompleteness. Our evaluation includes lexical KGs such as WN18RR, which capture structured linguistic hierarchies; encyclopedic KGs such as CoDEx-M and FB15K-237, which contain dense and heterogeneous relational patterns; and a socio-economic KG, which is sparse and semantically rich, derived from unstructured textual data. Although OMNIA was originally designed to recover missing triples in LLM-generated Knowledge Graphs, its architecture, which combines clustering, embedding-based filtering, and LLM-based validation, also allows it to handle incompleteness in traditional KGs. These results show that OMNIA generalizes well beyond its initial scope and adapts effectively to heterogeneous KG completion scenarios.

H. Efficiency

We evaluated the efficiency and scalability of OMNIA by measuring the average execution time of its main components on CoDEx-M and FB15k-237 samples of increasing sizes. Table VI reports the results for the candidate generation and embedding-based filtering steps. Candidate generation is the most time-consuming operation, with execution time increasing faster than linearly as dataset size grows, consistent with the complexity analysis presented above. However, since this step is performed offline, the cost remains acceptable. In contrast, the embedding filtering step exhibits a near-linear trend with a small and stable overhead. These results demonstrate that OMNIA maintains practical execution times and scales efficiently even for larger samples, confirming its suitability for medium and large-scale Knowledge Graphs.

Table VII reports the average execution time of the LLM-based evaluation across different sample sizes of candidates.

As expected, the execution time increases with the number of evaluated triples, reflecting the near-linear dependence of LLM inference on input size. Although the execution time appears long, since each evaluation is independent of the others, it can be significantly reduced by distributing the workload across multiple machines and nodes, as parallel processing enables efficient scaling.

Among all prompting strategies, zero-shot evaluation is the fastest for both triple-based and sentence-based inputs, because it does not require any context retrieval. In-context prompting introduces moderate overhead due to the inclusion of a small set of examples, while RAG incurs the highest computational cost because of its additional retrieval stage. For smaller sample sizes, the in-context strategy is notably more efficient than RAG. However, as the number of evaluated triples increases, their execution times converge, indicating that RAG's overhead remains constant and that its runtime scales almost linearly with dataset size. Overall, these results confirm that all prompting strategies exhibit consistent and scalable runtime behavior, runtime behavior, with RAG offering the best trade-off between contextual accuracy and computational efficiency.

TABLE VI: Average execution time (in seconds) of each step across different sample sizes (in thousands).

OMNIA's step	**Sample Size (in thousands)**				
	10	**20**	**30**	**40**	**50**
Candidate generation	6.08	26.47	72.42	160.24	304.83
Embedding filtering	13.59	15.16	19.47	24.30	30.53

TABLE VII: Average execution time (in seconds) of LLM-based evaluation for different sample sizes.

LLM Scenario	**Sample Size**			
	50	**100**	**250**	**500**
Triple-based Evaluation				
Zero-shot	85.18	181.87	532.52	1014.35
In-context	138.59	264.61	672.2	1481.35
RAG	567.76	687.24	987.33	1489.83
Sentence-based Evaluation				
Zero-shot	133.91	326.7	756.07	1517.59
In-context	300.53	687.56	1712.83	3365.93
RAG	743.64	1057.81	1852.87	3156.91

We further compared OMNIA with standard embedding approaches using a sample of 50,000 triples from the CoDEx-M and FB15K-237 datasets. This size was selected for consistency with our scalability analysis, ensuring a representative yet computationally feasible comparison. Traditional embedding models such as TransE, RotatE, ComplEx, and DistMult completed training and scoring in 35 seconds to 53 seconds on average. Although OMNIA is slower, it provides significant performance improvement. It increases the F1-score by 23% on CoDEx-M (from 0.68 to 0.91) and by 12% on FB15K-237 (from 0.74 to 0.86) compared to the best embedding-based baseline. These results confirm that OMNIA's higher computational cost directly translates into better semantic accuracy and higher-quality triple validation, making it particularly effective in precision-critical KG completion scenarios.

VI. Conclusion

In this paper, we introduce OMNIA, a two-stage approach for improving the completeness of knowledge graphs. OMNIA is broadly applicable to various types of KGs, while being particularly well suited to LLM-generated KGs, where some triples are semantically entailed by the source text but not explicitly extracted.

OMNIA first clusters entities that appear in similar relational contexts to generate candidate triples. These candidates are then filtered using a lightweight embedding-based model and validated by an LLM through different prompting strategies. We conducted extensive experiments across multiple datasets, demonstrating that OMNIA consistently improves triple classification performance over existing baselines, with gains of up to 23 percentage points in F1-score on several benchmarks.

Our results show that OMNIA is particularly effective on denser KGs, where structural patterns occur more frequently. On very sparse graphs, candidate generation becomes more challenging, as fewer valid candidates can be produced prior to validation, while the LLM-based validation remains effective once candidates are available. To address this limitation, we plan to improve candidate generation for sparse datasets. Possible directions include automatically detecting sparse regions of a KG and applying alternative strategies, such as semantic similarity or LLM-assisted candidate generation when structural signals are insufficient. These extensions would improve robustness to sparsity while preserving OMNIA's overall design.

Furthermore, we plan to include additional LLM-based baselines beyond our internal prompting variants and to evaluate OMNIA on a broader range of datasets, particularly KGs generated by LLMs, to further validate its adaptability to diverse KG construction contexts. We also intend to refine the filtering component to improve precision while maintaining efficiency, and to explore evaluation metrics better suited to the open-world nature of KGs.

Acknowledgments

This work is funded by the French National Agency ANR-21-CE23-0036-01 and he German Research Foundation under the project number 490998901.

Artifacts

The code of OMNIA is available on github via the link : https://github.com/fieng94/OMNIA.git

AI-Generated Content Acknowledgment

Some parts of this paper were prepared with the assistance of AI tools. These tools were used to support the writing process by improving clarity, coherence, and formatting. All scientific ideas, experimental designs, analyses, and interpretations were conceived, validated, and verified by the authors.

References

[1] Mehdi Ali et al. "PyKEEN 1.0: A Python Library for Training and Evaluating Knowledge Graph Embeddings". In: *Journal of Machine Learning Research* 22.82 (2021), pp. 1–6. URL: https://jmlr.org/papers/v22/20-825.html.

[2] Daniel Balouek et al. "Adding Virtualization Capabilities to the Grid'5000 Testbed". In: *Cloud Computing and Services Science*. Vol. 367. Communications in Computer and Information Science. Springer, 2013, pp. 3–20. DOI: 10.1007/978-3-319-04519-1_1. URL: https://link.springer.com/chapter/10.1007/978-3-319-04519-1_1.

[3] Russa Biswas et al. "Entity type prediction in knowledge graphs using embeddings". In: *arXiv preprint arXiv:2004.13702* (2020).

[4] Antoine Bordes et al. "Translating Embeddings for Modeling Multi-relational Data". In: *27th Annual Conference on Neural Information Processing Systems*. 2013, pp. 2787–2795.

[5] Antoine Bordes et al. "Translating embeddings for modeling multi-relational data". In: *Advances in neural information processing systems* 26 (2013).

[6] Tom Brown et al. "Language models are few-shot learners". In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.

[7] Harrison Chase. *LangChain*. https : / / github . com / langchain-ai/langchain. Accessed: 2025-06-04. 2022.

[8] Chen Chen et al. "Knowledge is flat: A seq2seq generative framework for various knowledge graph completion". In: *arXiv preprint arXiv:2209.07299* (2022).

[9] Tim Dettmers et al. "Convolutional 2d knowledge graph embeddings". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018.

[10] Morteza Kamaladdini Ezzabady et al. "Towards Generating High-Quality Knowledge Graphs by Leveraging Large Language Models". In: *International Conference on Applications of Natural Language to Information Systems*. Springer. 2024, pp. 455–469.

[11] Yuxia Geng et al. "Relational message passing for fully inductive knowledge graph completion". In: *2023 IEEE 39th international conference on data engineering (ICDE)*. IEEE. 2023, pp. 1221–1233.

[12] Wenbin Guo et al. "Ontology-Enhanced Knowledge Graph Completion using Large Language Models". In: *arXiv preprint arXiv:2507.20643* (2025).

[13] Elwin Huaman and Dieter Fensel. "Knowledge graph curation: a practical framework". In: *Proceedings of the 10th International Joint Conference on Knowledge Graphs*. 2021, pp. 166–171.

[14] Albert Q Jiang et al. "Mistral 7B". In: *arXiv preprint arXiv:2310.06825* (2023).

[15] Jeff Johnson, Matthijs Douze, and Hervé Jégou. "Billion-scale similarity search with GPUs". In: *IEEE Transactions on Big Data* 7.3 (2019), pp. 535–547.

[16] Hanieh Khorashadizadeh et al. "Construction and Canonicalization of Economic Knowledge Graphs with LLMs". In: *Knowledge Graphs and Semantic Web*. Cham: Springer Nature Switzerland, 2025, pp. 334–343. ISBN: 978-3-031-81221-7.

[17] Dawei Li et al. "Contextualization distillation from large language model for knowledge graph completion". In: *arXiv preprint arXiv:2402.01729* (2024).

[18] Qian Li et al. "LLM-based multi-level knowledge generation for few-shot knowledge graph completion". In: *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. Vol. 271494703. 2024.

[19] Yankai Lin et al. "Learning entity and relation embeddings for knowledge graph completion". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 29. 1. 2015.

[20] Christian Meilicke et al. *Reinforced Anytime Bottom Up Rule Learning for Knowledge Graph Completion*. 2020. arXiv: 2004.04412 [cs.AI]. URL: https://arxiv.org/abs/2004.04412.

[21] Changsung Moon, Paul Jones, and Nagiza F Samatova. "Learning entity type embeddings for knowledge graph completion". In: *Proceedings of the 2017 ACM on conference on information and knowledge management*. 2017, pp. 2215–2218.

[22] Heiko Paulheim. "Knowledge graph refinement: A survey of approaches and evaluation methods". In: *Semantic web* 8.3 (2017), pp. 489–508.

[23] *QualityOnt project*. URL: https : / / helios2 . mi . parisdescartes.fr/~sseifedd/qualityont.

[24] Arkadiy Saakyan, Tuhin Chakrabarty, and Smaranda Muresan. "COVID-fact: Fact extraction and verification of real-world claims on COVID-19 pandemic". In: *arXiv preprint arXiv:2106.03794* (2021).

[25] Tara Safavi and Danai Koutra. "Codex: A comprehensive knowledge graph completion benchmark". In: *arXiv preprint arXiv:2009.07810* (2020).

[26] Apoorv Saxena, Adrian Kochsiek, and Rainer Gemulla. "Sequence-to-Sequence Knowledge Graph Completion and Question Answering". In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, May 2022, pp. 2814–2828.

[27] Tong Shen, Fu Zhang, and Jingwei Cheng. "A comprehensive overview of knowledge graph completion". In: *Knowledge-Based Systems* 255 (2022), p. 109597.

[28] Baoxu Shi and Tim Weninger. "Open-world knowledge graph completion". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018.

[29] Dong Shu et al. "Knowledge graph large language model (KG-LLM) for link prediction". In: *arXiv preprint arXiv:2403.07311* (2024).

[30] Kaitao Song et al. "Mpnet: Masked and permuted pre-training for language understanding". In: *Advances*

in neural information processing systems 33 (2020), pp. 16857–16867.

[31] Zhiqing Sun et al. *RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space*. 2019. arXiv: 1902.10197 [cs.LG]. URL: https://arxiv.org/abs/1902.10197.

[32] Kristina Toutanova and Danqi Chen. "Observed versus latent features for knowledge base and text inference". In: *Proceedings of the 3rd workshop on continuous vector space models and their compositionality*. 2015, pp. 57–66.

[33] Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. *Enhancing Knowledge Graph Construction Using Large Language Models*. 2023. arXiv: 2305.04676 [cs.CL]. URL: https://arxiv.org/abs/2305.04676.

[34] Théo Trouillon et al. *Complex Embeddings for Simple Link Prediction*. 2016. arXiv: 1606.06357 [cs.AI]. URL: https://arxiv.org/abs/1606.06357.

[35] Théo Trouillon et al. "Complex embeddings for simple link prediction". In: *International conference on machine learning*. PMLR. 2016, pp. 2071–2080.

[36] Bo Wang et al. "Structure-augmented text representation learning for efficient knowledge graph completion". In: *Proceedings of the Web Conference 2021*. 2021, pp. 1737–1748.

[37] Liang Wang et al. "Simkgc: Simple contrastive knowledge graph completion with pre-trained language models". In: *arXiv preprint arXiv:2203.02167* (2022).

[38] Xiangyu Wang et al. "Knowledge graph quality control: A survey". In: *Fundamental Research* 1.5 (2021), pp. 607–626. ISSN: 2667-3258. DOI: https://doi.org/10.1016/j.fmre.2021.09.003. URL: https://www.sciencedirect.com/science/article/pii/S2667325821001655.

[39] Xiangyu Wang et al. "Knowledge graph quality control: A survey". In: *Fundamental Research* 1.5 (2021), pp. 607–626.

[40] Yanbin Wei et al. "Kicgpt: Large language model with knowledge in context for knowledge graph completion". In: *arXiv preprint arXiv:2402.02389* (2024).

[41] Zhengzheng Wei and James A. Esquivel. "Enhancing Knowledge Graph Completion with Retrieval-Augmented Generation Using Large Language Models". In: *2024 6th International Conference on Frontier Technologies of Information and Computer (ICFTIC)*. 2024, pp. 828–831. DOI: 10.1109/ICFTIC64248.2024.10912943.

[42] Xin Xie et al. "From discrimination to generation: Knowledge graph completion with generative transformer". In: *Companion Proceedings of the Web Conference 2022*. 2022, pp. 162–165.

[43] Derong Xu et al. "Multi-perspective improvement of knowledge graph completion with large language models". In: *arXiv preprint arXiv:2403.01972* (2024).

[44] Bishan Yang et al. *Embedding Entities and Relations for Learning and Inference in Knowledge Bases*. 2015. arXiv: 1412.6575 [cs.CL]. URL: https://arxiv.org/abs/1412.6575.

[45] Liang Yao, Chengsheng Mao, and Yuan Luo. *KG-BERT: BERT for Knowledge Graph Completion*. 2019. arXiv: 1909.03193 [cs.CL]. URL: https://arxiv.org/abs/1909.03193.

[46] Liang Yao, Chengsheng Mao, and Yuan Luo. "KG-BERT: BERT for knowledge graph completion". In: *arXiv preprint arXiv:1909.03193* (2019).

[47] Liang Yao et al. *Exploring Large Language Models for Knowledge Graph Completion*. 2025. arXiv: 2308.13916 [cs.CL]. URL: https://arxiv.org/abs/2308.13916.

[48] Yanpeng Ye et al. *Construction and Application of Materials Knowledge Graph in Multidisciplinary Materials Science via Large Language Model*. 2025. arXiv: 2404.03080 [cs.CL]. URL: https://arxiv.org/abs/2404.03080.

[49] Amrapali Zaveri et al. "Quality assessment for linked data: A survey". In: *Semantic Web* 7.1 (2016), pp. 63–93.

[50] Wen Zhang et al. "Start from zero: Triple set prediction for automatic knowledge graph completion". In: *IEEE Transactions on Knowledge and Data Engineering* (2024).

[51] Yichi Zhang et al. *Making Large Language Models Perform Better in Knowledge Graph Completion*. 2024. arXiv: 2310.06671 [cs.CL]. URL: https://arxiv.org/abs/2310.06671.

[52] Yujia Zhang et al. "Fine-tuning Language Models for Triple Extraction with Data Augmentation". In: *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models (KaLLM 2024)*. Ed. by Russa Biswas et al. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 116–124. DOI: 10.18653/v1/2024.kallm-1.12. URL: https://aclanthology.org/2024.kallm-1.12/.

[53] Yuqi Zhu et al. *LLMs for Knowledge Graph Construction and Reasoning: Recent Capabilities and Future Opportunities*. 2024. arXiv: 2305.13168 [cs.CL]. URL: https://arxiv.org/abs/2305.13168.